

Kolos 3

Operacje DDL

Tworzenie tabel (CREATE TABLE)

Składnia

```
CREATE TABLE nazwa_tabeli (  
    kolumna1    TYP1    [opcje_kolumny],  
    kolumna2    TYP2    [opcje_kolumny],  
    ...  
    [znakowanie_więzów],  
    [opcjonalne_definicje_PK_CK_INETCIEOP_IEUNIE]  
);
```

Przykład

```
CREATE TABLE users_user (  
    user_id      INT IDENTITY(1,1)    PRIMARY KEY,  
    email        VARCHAR(255)         NOT NULL    UNIQUE,  
    first_name   VARCHAR(200)         NOT NULL,  
    last_name    VARCHAR(200)         NOT NULL,  
    is_active    BIT                  NOT NULL    DEFAULT 1,  
    age          INT                  NULL,  
    current_balance MONEY             NOT NULL    DEFAULT 0  
);
```

Ograniczenia (Constraints)

1. **Primary key** - jednoznaczne identyfikowanie wierszy w tabeli, automatycznie zakłada `UNIQUE` i `NOT NULL`

```
CREATE TABLE course_enrollment (  
    user_id INT NOT NULL,  
    course_id INT NOT NULL,  
    ...  
    PRIMARY KEY (user_id) // albo PRIMARY KEY(user_id, course_id)  
)
```

2. **Foreign key** - wiązanie kolumn z kluczem głównym innej tabeli

```
// zał. tabela 'user' użytkowników i 'bicycles' rowerów  
CREATE TABLE bicycle_rent (  
    ...  
    user_id INT NOT NULL,  
    bicycle_id INT NOT NULL,  
    ...  
    FOREIGN KEY (user_id) REFERENCES user (user_id)  
    FOREIGN KEY (bicycle_id) REFERENCES bicycles (bicycle_id)  
);
```

```

user_id INT IDENTITY(1,1) PRIMARY KEY,
bicycle_id INT NOT NULL,
...
CONSTRAINT FK_rent_user
    FOREIGN KEY (user_id) REFERENCES users(user_id)
CONSTRAINT KF_rent_bicycle
    FOREIGN KEY (bicycle_id) REFERENCES bicycles(bicycle_id)
)

```

3. **UNIQUE** - wymusza unikalność w kolumnie

```

CREATE TABLE users_user (
    user_id INT IDENTITY(1,1) PRIMARY KEY,
    email VARCHAR(255) NOT NULL UNIQUE, // można dodać UNIQUE od razu przy tworzeniu
    ...
)

```

4. **NOT NULL** - oczywiste

5. **CHECK** - warunek logiczny, który musi być spełniony dla każdego wstawianego lub aktualizowanego wiersza. W przeciwnym przypadku zwróci błąd

```

CREATE TABLE courses (
    course_id INT IDENTITY(1,1) PRIMARY KEY,
    date_start DATE NOT NULL,
    date_end DATE NOT NULL,
    ...
    CONSTRAINT CHK_course_dates CHECK (date_start < date_end)
)

```

6. **DEFAULT** - ustawia domyślną wartość dla kolumny, jeśli podczas wstawiania nie podamy wartości

```

CREATE TABLE users (
    ...
    is_active BIT NOT NULL DEFAULT 1
)

```

Modyfikowanie table (ALTER TABLE)

1. **Dodawanie kolumny**

```

ALTER TABLE users_user
    ADD phone_number VARCHAR(25) NULL;

```

2. **Usuwanie kolumny** - jeśli kolumna była częścią więzu (np. `CHECK` lub `FOREIGN KEY`) to trzeba najpierw *usunąć ten constraint*, a dopiero potem usunąć kolumnę

```
ALTER TABLE
  DROP COLUMN age;
```

3. Dodawanie więzów (constraints)

```
// mamy tabelę 'courses' z kursami i 'course_enrollment' z informacjami o zapisach
ALTER TABLE course_enrollment
  ADD CONSTRAINT FK_enrollment_course
    FOREIGN KEY (course_id)
    REFERENCES courses(course_id)
```

4. Dodawanie unikalności

```
ALTER TABLE users
  ADD CONSTRAINT UQ_users_email
    UNIQUE (email)
```

5. Dodanie warunku (check)

```
ALTER TABLE course
  ADD CONSTRAINT CHK_course_dates
    CHECK (date_start < date_end);
```

6. Dodanie domyślnej wartości (DEFAULT)

```
ALTER TABLE groups
  ADD CONSTRAINT DF_group_max_capacity
    DEFAULT 10 FOR max_group_capacity
```

7. Usuwanie więzów

```
ALTER TABLE nazwa_tabeli
  DROP CONSTRAINT nazwa_więzu // np. DF_group_max_capacity
```

8. Zmiana typu lub właściwości kolumny

```
// w tabeli 'courses' kolumna course_name to NVARCHAR(100), chcemy zamienić na NVARCHAR(200)
ALTER TABLE courses
  ALTER COLUMN course_name NVARCHAR(200) NOT NULL;
```

Usuwanie obiektów (DROP)

1. **Usuwanie tabel** - jeśli tabela jest powiązana z innymi (np. przez `FOREIGN KEY`) to najpierw trzeba usunąć te powiązania

```
DROP TABLE IF EXISTS nazwa_tabeli
```

2. **Usuwanie indeksów** - potrzebujemy znać nazwę usuwanego indeksu

```
DROP INDEX IF EXISTS nazwa_indeksu ON nazwa_tabeli
```

Indeksy

Indeks - dodatkowa struktura (zwykle B-drzewo), która przyspiesza wyszukiwanie wierszy w tabeli. Bez indeksu przy każdym zapytaniu z filtrem `WHERE` lub `JOIN` baza musi przeskanować każdy wiersz tabeli, co jest wolne przy dużej ilości indeksów

Przykład - bez indeksu baza musi przeszukać wszystkie emaile, co jest wolne jak jest ich np. 100000

```
SELECT *  
FROM users  
WHERE email = 'kowalski@gmail.com'
```

Z indeksem na kolumnie `email` wyszukiwanie danego rekordu będzie w czasie logarytmicznym

```
CREATE UNIQUE INDEX idx_users_email  
ON users(email)
```

Rodzaje indeksów

Indeks klastrowany (Clustered index)

Fizycznie porządkuje wiersze tabeli na dysku według klucza indeksu - możliwy jest tylko jeden, w MS SQL domyślnie `PRIMARY KEY` tworzy indeks klastrowany

Stosujemy go gdy często filtrujemy lub sortujemy po danej kolumnie (np. `ORDER BY id`, `WHERE id = 123`)

Tworzenie

```
CREATE CLUSTERED INDEX idx_users_id  
ON users(user_id)
```

Indeks nieklastrowany (Non-clustered index)

Tworzy osobną strukturę danych (B-drzewo), w której przechowywane są wartości kolumny (kolumn) oraz wskaźnik do wiersza w tabeli właściwej. Można ich mieć wiele (np. na różnych kolumnach)

Tworzenie

```
CREATE NONCLUSTERED INDEX idx_users_lastname
ON users(last_name)
```

Dzięki niemu każde zapytanie z `WHERE last_name = 'Kowalski'` skorzysta z indeksu zamiast przeszukiwać całą tabelę

Indeks unikalny (Unique index)

Wymusza, że kombinacja wartości w indeksowanych kolumnach jest zawsze unikalna (tak jak `UNIQUE` w definicji kolumny)

Tworzenie

```
CREATE UNIQUE INDEX uidx_users_email
ON users(email)
```

Po stworzeniu indeksu próby wstawienia wiersza z tym samym emailiem spowodują błąd

Indeks złożony (Composite index)

Stosujemy, gdy w zapytaniach filtrujemy równocześnie po dwóch lub więcej kolumnach, a te kolumny są często używane w `WHERE`, `ORDER BY`, `GROUP BY`. Kolejność kolumn w definicji indeksu *ma znaczenie*, najpierw powinna być kolumna, po której częściej filtrujemy

Przykład - często robimy zapytania typu

```
SELECT *
FROM bicycle_rent
WHERE rent_station = @stacja
AND returned_at IS NOT NULL
```

Warto wtedy utworzyć indeks złożony `(rent_station, returned_at)`

```
CREATE NONCLUSTERED INDEX idx_bicycle_rent_station_returned
ON bicycle_rent(rent_station, returned_at)
```

Usuwanie indeksów

Musimy znać jego nazwę i tabelę, na której się znajduje

```
DROP INDEX IF EXISTS nazwa_indeksu ON nazwa_tabeli
```

Transakcje

Transakcja – logiczny ciąg operacji DB: atomowość, spójność, izolacja, trwałość (ACID).

- *Atomowość* - albo wszystkie instrukcje w transakcji będą zatwierdzone, albo żadna
- *Spójność* - jeśli przed rozpoczęciem transakcji baza była w stanie poprawnym (żadne więzy i constraints nie były naruszone), to po zatwierdzeniu transakcji również
- *Izolacja* - w trakcie wykonywania transakcji inne równoległe transakcje "nie widzą" jej pośrednich rezultatów. Dzięki temu unikamy zjawisk tj. "brudne odczyty" (*dirty reads*), "niepowtarzalne odczyty" (*non repeatable reads*) czy *phantom reads*
- *Trwałość* - gdy transakcja zostanie zatwierdzona (`COMMIT`), jej efekty zapisują się w taki sposób, że nawet awaria nie spowoduje ich utraty

Składnia (MS SQL):

```
BEGIN TRANSACTION;  
  -- operacje DML  
COMMIT;    -- zatwierdzenie  
ROLLBACK;  -- wycofanie
```

Przykład

Dodanie rekordu do `OrderDetails` i aktualizacja pola `UnitsInStock` dla `ProductID = 11` w tabeli `Products`

```
BEGIN TRANSACTION;  
BEGIN TRY  
  -- 1) INSERT do OrderDetails  
  INSERT INTO OrderDetails (OrderID, ProductID, Quantity)  
  VALUES (10248, 11, 5);  
  
  -- 2) UPDATE w Products  
  UPDATE Products  
    SET UnitsInStock = UnitsInStock - 5  
    WHERE ProductID = 11;  
  
  -- 3) zatwierdzamy  
  COMMIT;  
END TRY  
BEGIN CATCH  
  -- w razie wyjątku  
  ROLLBACK;  
  
  -- wypisanie błędu  
  DECLARE @ErrMsg NVARCHAR(4000) = ERROR_MESSAGE()  
  SELECT @ErrMsg AS ErrorMessage  
END CATCH
```

Poziomy izolacji

Wykorzystywane przy równoległych transakcjach, które określają jak jedna transakcja jest "odizolowana" od równoległych

1. **READ UNCOMMITTED** - Pozwala odczytywać dane, które zostały wstawione/zmienione wewnątrz innej transakcji, ale tej transakcji jeszcze nie zatwierdzono -> może dojść do "brudnych odczytów" (*dirty reads*)
2. **READ COMMITTED** - Domyślny w MS SQL, blokuje możliwość odczytu niezatwierdzonych zmian (czytamy dopiero to co zostało `COMMIT`-nięte)
 - Mogą występować "niepowtarzalne odczyty" (*non-repeatable reads*)
 - Przykład:

```
-- Transakcja A:
SELECT UnitsInStock FROM Products WHERE ProductID = 11; -- dostaje np. 100
-- (nie zatwierdzona jeszcze)

-- Transakcja B równolegle:
UPDATE Products SET UnitsInStock = UnitsInStock - 5 WHERE ProductID = 11;
COMMIT; -- zmiana (100 -> 95) jest teraz zatwierdzona

-- Transakcja A (kontynuacja):
SELECT UnitsInStock FROM Products WHERE ProductID = 11; -- dostanie już 95
-- W ramach tej samej transakcji A odczyty z tego samego wiersza mogą się różnić
```

3. **REPEATABLE READ** - zapewnia, że jeśli w transakcji A odczytaliśmy wiersz, to żadna inna równoległa transakcja nie może zmienić lub usunąć tego wiersza do czasu zakończenia transakcji A
 - Nadal mogą występować *phantom reads* (tzn. pojawienie się nowych wierszy pasujących do kryteriów, których nie było na początku)
4. **SERIALIZABLE** - najwyższy poziom izolacji, blokuje zmiany jak i dodawanie nowych wierszy pasujących do kryterium `WHERE`

Zmiana poziomu izolacji

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE; -- domyślnie READ COMMITTED
BEGIN TRANSACTION
    -- operacje DML
COMMIT;
```

Procedury składowane

Definicja

Procedura składowana - Blok kodu SQL przechowywany w bazie, wywoływany z przekazanymi parametrami

Składnia

```

CREATE PROCEDURE dbo.NazwaProcedury
    @Param1 INT, -- @p INT = 0 dla domyślnych wartości
    @Param2 NVARCHAR(50) OUTPUT
AS
BEGIN
    SET NOCOUNT ON; -- wyłączanie komunikatu np. (1 row(s) affected)
    -- ciało procedury: INSERT/UPDATE/DELETE/SELECT
END;

```

Przykłady

1. INSERT INTO Procedura `AddOrderDetail`, która przyjmuje `OrderID`, `ProductID` i `Quantity`, dodaje go do tabeli `OrderDetails` i zwraca w parametrze `OUTPUT` komunikat sukcesu albo błędu

```

CREATE PROCEDURE dbo.AddOrderDetail
    @OrderID INT,
    @ProductID INT,
    @Quantity INT,
    @Message NVARCHAR(100) OUTPUT
AS
BEGIN
    BEGIN TRY
        INSERT INTO OrderDetails (OrderID, ProductID, Quantity)
        VALUES (@OrderID, @ProductID, @Quantity);
        SET @Message = N'Wstawiono wiersz pomyślnie';
    END TRY
    BEGIN CATCH
        SET @Message = ERROR_MESSAGE();
    END CATCH
END;

-- Wywołanie:
DECLARE @Msg NVARCHAR(100);
EXEC dbo.AddOrderDetail
    @OrderID = 10248,
    @ProductID = 11,
    @Quantity = 5,
    @Message = @Msg OUTPUT;

SELECT @Msg AS Message

```

2. UPDATE Procedura ma przyjąć `OrderID`, `ProductID`, `NewQuantity` i zaktualizować wartości w tabeli `OrderDetails` oraz zwrócić komunikat w parametrze `OUTPUT`

```

CREATE PROCEDURE dbo.UpdateOrderDetailQuantity
    @OrderID INT,

```



```

@ProductID INT,
@NewQuantity INT,
@Message NVARCHAR(100) OUTPUT
AS
BEGIN
    BEGIN TRY
        UPDATE OrderDetails
        SET Quantity = @NewQuantity
        WHERE OrderID = @OrderID
            AND ProductID = @ProductID;

        IF @@ROWCOUNT = 0 -- systemowa zmienna
            SET @Message = 'Brak wiersza';
        ELSE
            SET @Message = 'OK';
    END TRY
    BEGIN CATCH
        SET @Message = ERROR_MESSAGE();
    END CATCH
END

```

3. DELETE Procedura przyjmuje `OrderID`, `ProductID` i usuwa odpowiadający wiersz z `OrderDetails` oraz zwraca komunikat w parametrze `OUTPUT`

```

CREATE PROCEDURE dbo.DeleteOrderDetail
    @OrderID INT,
    @ProductID INT,
    @Message NVARCHAR(100) OUTPUT
AS
BEGIN
    BEGIN TRY
        DELETE FROM OrderDetails
        WHERE OrderID = @OrderID
            AND ProductID = @ProductID;

        IF @@ROWCOUNT = 0
            SET @Message = N'Nie znaleziono wiersza do usunięcia';
        ELSE
            SET @Message = N'Wiersz usunięty pomyślnie';
    END TRY
    BEGIN CATCH
        SET @Message = ERROR_MESSAGE();
    END CATCH
END

```

4. UPDATE z REPLACE Procedura przyjmuje `Country`, `OldChar` (cyfra do zamiany), `NewChar` (na jaką cyfrę zamienić), modyfikuje numery telefonów w danym kraju i zwraca komunikat w parametrze `OUTPUT`

```

CREATE PROCEDURE dbo.UpdateCustomerPhoneByCountry
    @Country NVARCHAR(50),
    @OldChar CHAR(1),
    @NewChar CHAR(1),
    @Message NVARCHAR(100) OUTPUT
AS
BEGIN
    BEGIN TRY
        UPDATE Customers
        SET Phone = REPLACE(Phone, @OldChar, @NewChar)
        WHERE Country = @Country;

        IF @@ROWCOUNT = 0
            SET @Message = N'Brak klientów w kraju ' + @Country;
        ELSE
            SET @Message CAST(@@ROWCOUNT AS NVARCHAR(10))
                + N' numerów zmieniono pomyślnie';
    END TRY
    BEGIN CATCH
        SET @Message = ERROR_MESSAGE();
    END CATCH
END

```

Stare kolokwia

Kol. 1 - Wypożyczalnia rowerów

Zad 1a. Tworzenie tabel zgodnie z wymaganiami

```

-- 1) Tabela użytkowników
CREATE TABLE users_user (
    user_id          INT          IDENTITY(1,1) PRIMARY KEY,
    email            VARCHAR(255) NOT NULL      UNIQUE,
    first_name       VARCHAR(200) NOT NULL,
    last_name        VARCHAR(200) NOT NULL,
    is_active        BIT          NOT NULL      DEFAULT 1,
    age              INT          NULL,
    current_balance  MONEY        NOT NULL      DEFAULT 0
);

-- 2) Tabela stacji rowerowych
CREATE TABLE bicycle_station (
    station_id       INT          IDENTITY(1,1) PRIMARY KEY,
    city             VARCHAR(100) NOT NULL,
    address          VARCHAR(255) NOT NULL,
    racks_amount     INT          NOT NULL,
    available_racks  INT          NOT NULL,
    CONSTRAINT CHK_station_racks CHECK (available_racks BETWEEN 0 AND racks_amount)
)

```

```

);

-- 3) Tabela rowerów
CREATE TABLE bicycle (
    rover_id          INT          IDENTITY(1,1) PRIMARY KEY,
    code              VARCHAR(50)  NOT NULL    UNIQUE,
    is_available      BIT          NOT NULL    DEFAULT 1,
    current_station   INT          NOT NULL,
    price_per_hour    DECIMAL(10,2) NOT NULL,
    CONSTRAINT FK_bicycle_station
        FOREIGN KEY (current_station) REFERENCES bicycle_station(station_id)
);

-- 4) Tabela wypożyczeń rowerów
CREATE TABLE bicycle_rent (
    rent_id           INT          IDENTITY(1,1) PRIMARY KEY,
    user_id           INT          NOT NULL,
    bicycle_id        INT          NOT NULL,
    rented_at         DATETIME     NOT NULL    DEFAULT GETDATE(),
    returned_at       DATETIME     NULL,
    rent_station      INT          NOT NULL,
    return_station    INT          NULL,
    total_cost        DECIMAL(10,2) NULL,
    is_finished       BIT          NOT NULL    DEFAULT 0,
    is_returned_manually BIT      NOT NULL    DEFAULT 0,

    CONSTRAINT FK_rent_user FOREIGN KEY (user_id)
        REFERENCES users_user(user_id),
    CONSTRAINT FK_rent_bicycle FOREIGN KEY (bicycle_id)
        REFERENCES bicycle(rover_id),
    CONSTRAINT FK_rent_rentstation FOREIGN KEY (rent_station)
        REFERENCES bicycle_station(station_id),
    CONSTRAINT FK_rent_returnstation FOREIGN KEY (return_station)
        REFERENCES bicycle_station(station_id)
);

```

Zad 1b. Modyfikacja tabel

```

-- Dodanie kolumny phone_number (VARCHAR(25))
ALTER TABLE users_user
    ADD phone_number VARCHAR(25) NULL;

-- Dodanie kolumny birth_date (DATETIME) i constraint wymuszający wiek ≥ 18 lat
ALTER TABLE
    ADD birth_date DATETIME NULL;

ALTER TABLE users_user
    ADD CONSTRAINT CHK_users_birthdate_adult
        CHECK (DATEDIFF(YEAR, birth_date, GETDATE()) >= 18)

```

```
-- Usunięcie kolumny age
ALTER TABLE users_suer
    DROP COLUMN age;
```

Zad 1c. Dodawanie rekordów do tabeli

```
INSERT INTO users_user (email, first_name, last_name, is_acitve, current_balance, birth_date,
phone_number)
VALUES
    ('anna.kowalska@gmail.com', 'Anna', 'Kowalska', 1, 100.00, '1990-05-12', '600-123-456'),
    ('jan.nowak@gmail.com', 'Jan', 'Nowak', 1, 50.00, '1985-11-03', '600-234-567'),
    ('piotr.zielinski@example.com', 'Piotr', 'Zielinski', 0, 20.00, '2000-07-22', '600-345-
678');
```

```
INSERT INTO bicycle_station (city, address, racks_amount, available_racks)
VALUES
    ('Warsaw', 'Ul. Marszałkowska 10', 20, 5),
    ('Krakow', 'ul. Floriańska 15', 10, 2),
    ('Wroclaw', 'ul. Świdnicka 22', 15, 10);
```

```
INSERT INTO bicycle (code, is_available, current_station, price_per_hour)
VALUES
    ('R-202-X', 1, 1, 7.50),
    ('R-303-Y', 1, 1, 8.00),
    ('R-404-Z', 1, 2, 6.50);
```

```
-- Anna (id = 1), rower (id = 1), wczoraj o 14:00
```

```
INSERT INTO bicycle_rent (user_id, bicycle_id, rented_at, rent_station, is_finished,
is_returned_manually)
```

```
VALUES
    (1, 1, DATEADD(DAY, -1, CONVERT(DATETIME, CONVERT(DATE, GETDATE))) ) + '14:00', 1, 0, 0);
```

```
-- Jan (id = 1), rower (id = 2), dziś 2 godziny temu
```

```
INSERT INTO bicycle_rent (user_id, bicycle_id, rented_at, rent_station, is_finished,
is_returned_manually)
```

```
VALUES
    (2, 2, DATEADD(HOUR, -2, GETDATE()), 1, 0, 0);
```

```
-- ANNA (id = 1), rower (id = 3), dziś 3 godziny temu
```

```
INSERT INTO bicycle_rent (user_id, bicycle_id, rented_at, rent_station, is_finished,
is_returned_manually)
```

```
VALUES
    (1, 3, DATEADD(HOUR, -3, GETDATE()), -2, 0, 0);
```

Zad 2. Indeksy

1. Częste łączenie tabel `users_user` i `bicycle_rent`

Łączymy je za pomocą `user_id`, więc zbudujemy na jego podstawie indeks

```
CREATE NONCLUSTERED INDEX idx_bicycle_rent_user
ON bicycle_rent(user_id)
```

2. Wymuszenie unikalności `users_user.email`

```
CREATE UNIQUE INDEX uidx_users_email
ON users_user(email);
```

3. Filtracja w `bicycle_rent` według `rent_station` i `returned_at`

```
CREATE NONCLUSTERED INDEX idx_bicycle_rent_station_returned
ON bicycle_rent(rent_station, returned_at)
```

4. Złożony indeks w tabeli `bicycle_station`

Warto rozważyć `city`, `available_racks` i `address`, by szybko móc filtrować po miastach, dostępności stacji i adresów wewnątrz miasta

```
CREATE NONCLUSTERED INDEX idx_bicycle_station_city_racks
ON bicycle_station(city, available_racks);
```

5. Filtrowanie użytkowników według `first_name` i `last_name`

```
CREATE NONCLUSTERED INDEX idx_users_name
ON users_user(first_name, last_name);
```

Zad 3. Procedura składowana

Procedura składowana do zwrotu roweru wypożyczonego przez użytkownika

Procedura przyjmuje `@user_id`, `@rent_id`, `@station_id`

Oblicza koszt przejazdu zgodnie z zasadami:

- Przejazd krótszy niż **20** minut - **bezpłatny**
- Przejazd krótszy niż **60** minut - **1 zł**
- Przejazd krótszy niż **120** minut - **3 zł**
- Przejazd **≥ 120** minut - **7 zł/godzinę** (zaokrąglając w górę do pełnej godziny)

Sprawdza, czy jest dostępny stojak na podanej stacji (`available_racks`). Jeśli co najmniej jeden wolny stojak (`available_racks > 0`), to:

- Procedura **aktualizuje wiersz** w `bicycle_rent`, ustawiając `returned_at`, `return_station`, `total_cost`, `is_finished` = 1 (i `is_returned_manually` = 0)
- **Zmniejsza o 1** wartość `available_racks` w tabeli `bicycle_station`
- **Zmniejsza bieżące saldo użytkownika** w `users_user.current_balance` o wartość `total_cost`

Jeśli nie ma wolnych stojaków (`available_racks` = 0), to:

- Procedura **wstawia** w `bicycle_rent` datę zwrotu (`returned_at`), stację zwrotu (`returned_station`), `total_cost`, `is_finished` = 1 i `is_returned_manually` = 1
- **Nie zmienia** `available_racks`
- **Zmniejsza bieżące saldo użytkownika** `current_balance` o wartość `total_cost`

Rozwiązanie

```
CREATE PROCEDURE dbo.usp_ReturnBicycle
    @user_id INT,
    @rent_id INT,
    @station_id INT,
    @Message NVARCHAR(255) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    -- Deklarowanie pomocniczych zmiennych
    DECLARE
        @RentedAt DATETIME,
        @BicycleID INT,
        @RentStation INT,
        @DurationMin INT,
        @TotalCost MONEY
        @PricePerHour MONEY,
        @FreeRacks INT;

    BEGIN TRANSACTION;
    BEGIN TRY
        -- Weryfikacja, czy jest aktywne wypożyczenie (is_finished = 0)
        IF NOT EXISTS (
            SELECT 1
            FROM bicycle_rent
            WHERE rent_id = @rent_id
                AND is_finished = 0
        )
        BEGIN
            SET @Message = N'No active user found with ID = ' + CAST(@rent_id AS
            NVARCHAR(20))
            ROLLBACK;
            RETURN;
        END;
    END TRY
```

```

-- Pobranie dane dot. wypożyczenia: rented_at, bicycle_id, rent_station
SELECT
    @RentedAt = rented_at,
    @BicycleId = bicycle_id,
    @RentStation = rent_station
FROM bicycle_rent
WHERE rent_id = @rent_id

-- Obliczanie liczby minut trwania przejazdu
SET @DurationMin = DATEDIFF(MINUTE, @RentedAt, GETDATE());

-- Obliczanie kosztu przejazdu
IF @DurationMin < 20
    SET @TotalCost = 0;
ELSE IF @DurationMin < 60
    SET @TotalCost = 1;
ELSE IF @DurationMin < 120
    SET @TotalCost = 3
ELSE
BEGIN
    -- Pobieranie ceny za godzinę
    SELECT @PricePerHour = price_per_hour
    FROM bicycle
    WHERE rover_id = @BicycleID;

    -- Ile pełnych godzin
    DECLARE @HoursUsed INT = CEILING(@DurationMin / 60.0);

    SET @TotalCost = @HoursUsed * @PricePerHour;
END;

-- Sprawdzanie liczby wolnych stojaków
SELECT @FreeRacks = available_racks
FROM bicycle_station
WHERE station_id = @stationId;

IF @FreeRacks IS NULL
BEGIN
    SET @Message = N'Station with ID = ' + CAST(@station_id AS NVARCHAR(10)) + ' does
not exist';
    ROLLBACK;
    RETURN;
END;

IF @FreeRacks > 0
BEGIN
    UPDATE bicycle_rent
    SET
        returned_at = GETDATE(),
        return_station = @station_id,

```

```

        total_cost = @TotalCost,
        is_finished = 1,
        is_returned_manually = 0
WHERE rent_id = @rent_id

UPDATE bicycle_station
SET available_racks = available_racks - 1
WHERE station_id = @station_id;

UPDATE users_user
SET current_balance = current_balance - @TotalCost
WHERE user_id = @user_id;

SET @Message = N'Return successful. Cost: ' + CAST(@TotalCost AS NVARCHAR(20)) +
' zł.';
END
ELSE
BEGIN
    -- Free racks == 0
    UPDATE bicycle_rent
    SET
        returned_at = GETDATE(),
        return_station = @station_id,
        total_cost = @TotalCost,
        is_finished = 1,
        is_returned_manually = 1
    WHERE rent_id = @rent_id;

    UPDATE users_user
    SET current_balance = current_balance - @TotalCost
    WHERE user_id = @user_id;

    SET @Message = N'No racks left - manual return. Cost: ' + CAST(@TotalCost AS
    NVARCHAR(20)) + 'zł.';
    END;
    COMMIT;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK; -- sprawdzanie czy otwarta jest jakaś transakcja
    SET @Message = ERROR_MESSAGE();
END CATCH;
END

```

Kol. 2 - Kursy

Zad 1a. Tworzenie tabel zgodnie z wymaganiami

Wymagania


```
// model danych
```

Tabela: course

course_id	INT IDENTITY(1,1)	PRIMARY KEY
course_name	NVARCHAR(100)	NOT NULL
base_price	MONEY	NOT NULL
planned_groups_amount	INT	NOT NULL DEFAULT 1
date_start	DATE	NOT NULL
date_end	DATE	NOT NULL
is_active	BIT	NOT NULL DEFAULT 1

Tabela: users_user

user_id	INT IDENTITY(1,1)	PRIMARY KEY
email	NVARCHAR(255)	NOT NULL UNIQUE
first_name	NVARCHAR(200)	NOT NULL
last_name	NVARCHAR(200)	NOT NULL
is_active	BIT	NOT NULL DEFAULT 1
age	INT	NULL
current_balance	MONEY	NOT NULL DEFAULT 0

Tabela: course_enrollment

user_id	INT	NOT NULL	→ FK → users_user(user_id)
course_id	INT	NOT NULL	→ FK → course(course_id)
enrollment_date	DATETIME	NOT NULL	
total_cost	MONEY	NOT NULL	
discount_type	VARCHAR(100)	NOT NULL	DEFAULT 'bezwarynkowy'
discount_value	MONEY	NOT NULL	DEFAULT 0
is_completed	BIT	NOT NULL	DEFAULT 0
is_dropped	BIT	NOT NULL	DEFAULT 0
PRIMARY KEY (user_id, course_id)			

Tabela: [group]

group_id	INT IDENTITY(1,1)	PRIMARY KEY	
group_type	NVARCHAR(25)	NOT NULL	DEFAULT 'zajęciowa'
course_id	INT	NOT NULL	→ FK → course(course_id)
max_group_capacity	INT	NOT NULL	

Tabela: group_timetable

group_id	INT	NOT NULL	→ FK → [group](group_id)
room	NVARCHAR(10)	NOT NULL	
datetime_start	DATETIME	NOT NULL	
datetime_end	DATETIME	NOT NULL	
PRIMARY KEY (group_id, room, datetime_start)			

Rozwiązanie

```
CREATE TABLE course (
    course_id          INT          IDENTITY(1,1) PRIMARY KEY,
    course_name        NVARCHAR(100) NOT NULL,
    base_price         MONEY        NOT NULL,
    planned_groups_amount INT      NOT NULL DEFAULT 1,
    date_start         DATE         NOT NULL,
    date_end           DATE         NOT NULL,
    is_active          BIT          NOT NULL DEFAULT 1
);

CREATE TABLE users_user (
    user_id            INT          IDENTITY(1,1) PRIMARY KEY,
    email              NVARCHAR(255) NOT NULL UNIQUE,
    first_name         NVARCHAR(200) NOT NULL,
    last_name          NVARCHAR(200) NOT NULL,
    is_active          BIT          NOT NULL DEFAULT 1,
    age                INT          NULL,
    current_balance    MONEY        NOT NULL DEFAULT 0
);

CREATE TABLE course_enrollment (
    user_id            INT          NOT NULL,
    course_id          INT          NOT NULL,
    enrollment_date    DATETIME     NOT NULL,
    total_cost         MONEY        NOT NULL,
    discount_type      VARCHAR(100) NOT NULL DEFAULT 'bezwarunkowy',
    discount_value     MONEY        NOT NULL DEFAULT 0,
    is_completed       BIT          NOT NULL DEFAULT 0,
    is_dropped         BIT          NOT NULL DEFAULT 0,
    CONSTRAINT PK_course_enrollment PRIMARY KEY (user_id, course_id),
    CONSTRAINT FK_enrollment_user   FOREIGN KEY (user_id) REFERENCES users_user (user_id),
    CONSTRAINT FK_enrollment_course FOREIGN KEY (course_id) REFERENCES course
(course_id)
);

CREATE TABLE [group] (
    group_id           INT          IDENTITY(1,1) PRIMARY KEY,
    group_type         NVARCHAR(25) NOT NULL DEFAULT 'zajęciowa',
    course_id          INT          NOT NULL,
    max_group_capacity INT          NOT NULL,
    CONSTRAINT FK_group_course FOREIGN KEY (course_id) REFERENCES course(course_id)
);

CREATE TABLE group_timetable (
    group_id           INT          NOT NULL,
    room              NVARCHAR(10) NOT NULL,
    datetime_start     DATETIME     NOT NULL,
```

```

datetime_end    DATETIME    NOT NULL,
CONSTRAINT PK_group_timetable PRIMARY KEY (group_id, room, datetime_start),
CONSTRAINT FK_timetable_group FOREIGN KEY (group_id) REFERENCES [group](group_id)
);

```

Zad 1b. Modyfikacja tabel

```

-- Dodanie kolumny phone_number (VARCHAR(25)) do users_user
ALTER TABLE users_user
    ADD phone_number VARCHAR(25) NULL;

-- Dodanie kolumny birth_date (DATETIME) i sprawdzenie, że użytkownik ma ≥ 18 lat
ALTER TABLE users_user
    ADD birth_date DATETIME NULL;

ALTER TABLE users_user
    ADD CONSTRAINT CHK_users_birthdate_adult
        CHECK (DATEDIFF(YEAR, birth_date, GETDATE()) >= 18);

-- Usunięcie kolumny age z tabeli users_user
ALTER TABLE users_user
    DROP COLUMN age;

-- Dodanie CHECK w tabeli course, aby date_start < date_end
ALTER TABLE course
    ADD CONSTRAINT CHK_course_dates
        CHECK (date_start < date_end)

```

Zad 1c. Dodanie rekordów do tabeli

```

INSERT INTO users_user (email, first_name, last_name, is_active, current_balance, birth_date,
phone_number)
VALUES
    ('anna.kowalska@edu.com', 'Anna', 'Kowalska', 1, 500.00, '1995-02-10', '501-111-222'),
    ('jan.nowak@edu.com', 'Jan', 'Nowak', 1, 250.00, '1988-07-25', '502-222-333'),
    ('ewa.wozniak@edu.com', 'Ewa', 'Woźniak', 1, 0.00, '2002-12-05', '503-333-444');

INSERT INTO course (course_name, base_price, planned_groups_amount, date_start, date_end,
is_active)
VALUES
    ('Programowanie w C#', 1200.00, 2, '2023-09-01', '2023-12-01', 1),
    ('Bazy Danych - SQL', 1000.00, 3, '2023-10-15', '2024-02-15', 1),
    ('Wprowadzenie do Pythona', 800.00, 1, '2023-11-01', '2024-01-31', 0);

-- Anna (user_id=1) zapisała się na "Programowanie w C#" (course_id=1)
-- Jan (user_id=2) zapisał się na "Bazy Danych - SQL" (course_id=2)
-- Ewa (user_id=3) zapisała się na "Programowanie w C#" (course_id=1)
-- Wartość total_cost obliczamy z base_price (brak zniżek), discount_type domyślny

```

```

INSERT INTO course_enrollment (user_id, course_id, enrollment_date, total_cost,
discount_type, discount_value, is_completed, is_dropped)
VALUES
    (1, 1, GETDATE(), 1200.00, 'bezwarunkowy', 0.00, 0, 0),
    (2, 2, GETDATE(), 1000.00, 'bezwarunkowy', 0.00, 0, 0),
    (3, 1, GETDATE(), 1200.00, 'bezwarunkowy', 0.00, 0, 0),

-- Grupa 1 dla kursu 1, maks. 10 uczestników
-- Grupa 2 dla kursu 1, maks. 8 uczestników
-- Grupa 3 dla kursu 2, maks. 12 uczestników
INSERT INTO [group] (group_type, course_id, max_group_capacity)
VALUES
    ('zajęciowa', 1, 10),
    ('zajęciowa', 1, 8),
    ('zajęciowa', 2, 12);

-- Grupa 1 (group_id=1): sala A1 w dniach '2023-09-02 10:00' → '2023-09-02 12:00'
-- Grupa 2 (group_id=2): sala B2 w dniach '2023-09-03 14:00' → '2023-09-03 16:00'
-- Grupa 3 (group_id=3): sala C3 w dniu '2023-10-01 09:00' → '2023-10-01 11:00'
INSERT INTO group_timetable (group_id, room, datetime_start, datetime_end)
VALUES
    (1, 'A1', '2023-09-02 10:00', '2023-09-02 12:00'),
    (2, 'B2', '2023-09-03 14:00', '2023-09-03 16:00'),
    (3, 'C3', '2023-10-01 09:00', '2023-10-01 11:00');

```

Zad 2. Indeksy

1. Częste łączenie tabel users_user i course_enrollment

```

CREATE NONCLUSTERED INDEX idx_course_enrollment_user
ON course_enrollment(user_id);

```

2. Unikalność kolumny email w tabeli users_user

```

CREATE UNIQUE INDEX uidx_users_email
ON users_user(email);

```

3. Częste wyszukiwanie wg daty rozpoczęcia i zakończenia kursu

```

CREATE NONCLUSTERED INDEX idx_course_dates
ON course(date_start, date_end);

```

4. Złożony indeks w tabeli course_enrollment

Warto rozważyć `course_id` i `enrollment_date` by szybko móc filtrować po rozpoczęciach kursów

```
CREATE NONCLUSTERED INDEX_ idx_course_enrollment_course_date
ON course_enrollment(course_id, enrollment_date);
```

5. Filtracja użytkowników według first_name i last_name

```
CREATE NONCLUSTERED INDEX idx_users_name
ON users_user(first_name, last_name);
```

Zad 3. Procedura składowana

Procedura składowana do zapisu użytkownika na wybrany kurs

Procedura przyjmuje @email i @course_id

Wykonuje **walidację**

- Czy podany **użytkownik jest aktywny**
- Czy **kurs jest aktywny**
- Czy jest chociaż **jedno dostępne miejsce** w grupach przypisanych do danego kursu
- Jeśli **nie istnieje użytkownik** o podanym emailu, trzeba go utworzyć

Jeśli parametry spełniły walidację, to procedura oblicza koszt kursu

- Użytkownik kupił **pierwszy kurs** w systemie - **bezwarunkowy rabat 100zł**
- Użytkownik kupił **druki kurs** w systemie - **stały rabat 5%**
- Użytkownik kupił **n-ty kurs** ($n > 2$) - rabat lojalnościowy **n%**, który trzeba **dodać do stałego rabatu**

Procedura tworzy nowy rekord w tabeli course_enrollment

Rozwiązanie

```
CREATE PROCEDURE dbo.usp_EnrollInCourse
    @Email VARCHAR(255),
    @CourseID INT,
    @Message NVARCHAR(255) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    -- Deklarowanie pomocniczych zmiennych
    DECLARE
        @UserID INT,
        @IsActiveUser BIT,
        @IsActiveCourse BIT,
        @ExistingEnrolls INT,
        @BasePrice MONEY,
        @DiscountVal DECIMAL(10, 4),
        @DiscountType VARCHAR(50),
```

```

        @TotalCost MONEY,
        @AvailableSlots INT;

BEGIN TRANSACTION;
BEGIN TRY
    -- Sprawdzanie istnienia użytkownika wg email
    SELECT
        @UserID = user_id,
        @IsActiveUser = is_active
    FROM users_user
    WHERE email = @Email;

    IF @UserID IS NULL
    BEGIN
        -- Użytkownik nie istnieje
        INSERT INTO users_user (email, first_name, last_name, is_active, current_balance)
        VALUES (
            @Email,
            'Unknown', -- first_name
            'Unknown', -- last_name
            1, -- nowy użytkownik jest aktywny
            0 -- początkowe saldo
        );

        SET @UserID = SCOPE_IDENTITY(); -- UserID będzie automatycznie dopisane wg
IDENTITY(1,1)
        SET @IsActiveUser = 1;
    END
    ELSE IF @IsActiveUser = 0
    BEGIN
        -- Użytkownik istnieje, ale jest nieaktywny
        SET @Message = N'User with given username is inactive'
        ROLLBACK;
        RETURN;
    END;

    -- Sprawdzenie, czy kurs istnieje i jest aktywny
    SELECT
        @IsActiveCourse = is_active
        @BasePrice = base_price
    FROM course
    WHERE course_id = @CourseID;

    IF @IsActiveCourse IS NULL
    BEGIN
        SET @Message = N'Course with ID = ' + CAST(@CourseID AS NVARCHAR(10)) + N' does
not exist.';
        ROLLBACK;
        RETURN;
    END

```

```

ELSE IF @IsActiveCourse = 0
BEGIN
    SET @Message = N'Course is inactive.';
    ROLLBACK;
    RETURN;
END;

-- Sprawdzenie, ile jest wolnych miejsc w kursie
DECLARE @TotalCapacity INT;
DECLARE @AlreadyEnrolled INT;

SELECT
    @TotalCapacity = c.planned_groups_amout * SUM(g.max_group_capacity)
FROM course AS c
JOIN [group] AS g ON c.course_id = g.course_id
WHERE c.course_id = @CourseID
GROUP BY c.planned_groups_amount;

-- Podliczenie już istniejących zapisów
SELECT
    @AlreadyEnrolled = COUNT(*)
FROM course_enrollment
WHERE course_id = @CourseID
    AND is_dropped = 0;

SET @AvailableSlots = @TotalCapacity - @AlreadyEnrolled;

IF @AvailableSlots < 1
BEGIN
    SET @Message = N'No slots left in the course.';
    ROLLBACK;
    RETURN;
END;

-- Obliczanie wartości DISCOUNT i TOTAL_COST na podstawie liczby poprzednich zapisów
SELECT
    @ExistingEnrolls = COUNT(*)
FROM course_enrollment
WHERE user_id = @UserID
    AND is_dropped = 0;

IF @ExistingEnrolls = 0
BEGIN
    -- Pierwszy zakup - bezwarunkowy rabat 100zł
    SET @DiscountVal = 100.00;
    SET @DiscountType = N'bezwarunkowy'
    SET @TotalCost = @BasePrice - @DiscountVal;
    IF @TotalCost < 0 SET @TotalCost = 0;
END
ELSE IF @ExistingEnrolls = 1

```

```

BEGIN
    -- Drugi zakup - stały rabat 5%
    SET @DiscountVal = 0.05;
    SET @DiscountType = N'staly';
    SET @TotalCost = @BasePrice * (1.0 - @DiscountVal)
END
ELSE
BEGIN
    -- >2 zakup - lojalnościowy rabat 0.05 + (n * 0.01)
    SET @DiscountVal = 0.05 + (@ExistingEnrolls * 0.01);
    SET @DiscountType = N'lojalnosciowy';
    SET @TotalCost = @BasePrice * (1.0 - @DiscountVal);
    IF @TotalCost < 0 SET @TotalCost = 0;
END;

-- Wstawianie nowego zapisu do course_enrollment
INSERT INTO course_enrollment (
    user_id, course_id, enrollment_date, total_cost, discount_value, is_completed,
is_dropped
)
VALUES (
    @UserID,
    @CourseID,
    GETDATE(),
    @TotalCost,
    @DiscountType,
    @DiscountVal,
    0, -- is_completed
    0 -- is_dropped
);

SET @Message = N'Course enrollment successful. Cost: ' + CAST(@TotalCost AS
NVARCHAR(20)) + N' zł.';
COMMIT;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK;
    SET @Message = ERROR_MESSAGE();
END CATCH;
END;

```